

Computer Science Concepts Cheat Sheet

Quick Revision for Interviews, University Exams & Placement Tests

Contents

1. Java	1
2. Discrete Mathematics	4
3. Digital Logic Design	5
4. Numerical Methods	6
5. Automata and Computability	8
6. Computer Architecture	9
7. Compiler Design	10
8. Computer Graphics	11
9. Software Engineering	12
Last-Minute Cross-Subject Hit List	14

1. Java

Core Language Facts

- **Platform independent:** source $\rightarrow$ bytecode (.class) $\rightarrow$ JVM interprets/JIT-compiles per platform. *"Write once, run anywhere."*
- **JDK $\supset$ JRE $\supset$ JVM:** JDK = compiler + tools; JRE = JVM + libraries; JVM = execution engine.
- Java is **pass-by-value always** — object references are passed *by value* (classic interview trap).
- No pointers, no multiple inheritance of classes (use interfaces), automatic garbage collection.
- String is **immutable**; use `StringBuilder` (unsynchronized, fast) or `StringBuffer` (synchronized) for mutation.

OOP Pillars (Mnemonic: A PIE)

Pillar	Meaning	Java mechanism
Abstraction	Hide implementation, expose behavior	abstract classes, interfaces
Polymorphism	One interface, many forms	Overriding (runtime), Overloading (compile-time)

Pillar	Meaning	Java mechanism
Inheritance	Reuse via is-a relationship	extends, implements
Encapsulation	Data hiding + controlled access	private fields + getters/setters

Overloading vs Overriding

	Overloading	Overriding
Binding	Compile-time (static)	Runtime (dynamic dispatch)
Signature	Same name, different params	Same name, same params
Return type	Can differ freely	Same or covariant
Where	Same class	Subclass
static/private/final	Can be overloaded	Cannot be overridden (static = hidden)

Key Keywords

- **final**: variable → constant; method → no override; class → no subclass.
- **static**: belongs to class, not instance; static methods can't use `this/super`.
- **abstract class**: can have constructors, state, concrete methods; **cannot be instantiated**.
- **Interface (Java 8+)**: default and static methods allowed; fields are implicitly public static final; supports multiple inheritance of type.

Abstract Class vs Interface

	Abstract class	Interface
Inheritance	Single	Multiple
State	Instance fields allowed	Only constants
Constructor	Yes	No
Use when	Shared base implementation	Capability contract (e.g., Comparable)

Exception Hierarchy

Throwable

|-- Error (unrecoverable: OutOfMemoryError, StackOverflowError)

+-- Exception

|-- Checked (compile-time enforced: IOException, SQLException)

+-- RuntimeException (unchecked: NPE, ArrayIndexOutOfBoundsException, ClassCast)

- `try` → `catch` (specific first) → `finally` (always runs, even after return; skipped only on `System.exit`).
- `throw` = raise one exception; `throws` = declare in signature.
- `try-with-resources` auto-closes `AutoCloseable` (preferred over manual `finally close`).

Collections Framework (know complexities!)

Class	Backing	get	add	contains	Ordered?	Nulls
ArrayList	dynamic array	O(1)	O(1) amortized	O(n)	insertion	yes
LinkedList	doubly linked	O(n)	O(1) at ends	O(n)	insertion	yes
HashMap	array + buckets (tree bins ≥ 8)	O(1) avg	O(1) avg	O(1) avg	none	1 null key
LinkedHashMap	hash + linked list	O(1)	O(1)	O(1)	insertion/access	yes
TreeMap	Red-Black tree	O(log n)	O(log n)	O(log n)	sorted keys	no null key
HashSet	HashMap inside	—	O(1) avg	O(1) avg	none	1 null
TreeSet	TreeMap inside	—	O(log n)	O(log n)	sorted	no
PriorityQueue	binary heap	peek O(1)	O(log n)	O(n)	heap order	no

- **HashMap vs Hashtable:** Hashtable is legacy, synchronized, no nulls. Use ConcurrentHashMap for thread safety.
- **equals()/hashCode() contract:** equal objects $\Rightarrow$ equal hash codes. Override both or hash collections break (very common interview question).
- **Fail-fast** iterators (ArrayList) throw ConcurrentModificationException; **fail-safe** (CopyOnWriteArrayList, ConcurrentHashMap) iterate a snapshot.
- Comparable = natural order, compareTo(), in the class itself. Comparator = external, multiple orderings.

Multithreading Essentials

- Create: extend Thread or (preferred) implement Runnable / Callable (returns value, throws) + ExecutorService.
- Thread states: New $\rightarrow$ Runnable $\rightarrow$ Running $\rightarrow$ (Blocked/Waiting/Timed-Waiting) $\rightarrow$ Terminated.
- synchronized = mutual exclusion + visibility; volatile = visibility only (no atomicity).
- wait()/notify() must be called inside a synchronized block on the same monitor; wait() releases the lock, sleep() does not.
- Deadlock needs: mutual exclusion, hold & wait, no preemption, circular wait.

Memory & GC

- **Stack:** per-thread frames, local vars, references. **Heap:** all objects (young gen: Eden + survivors; old gen).
- **Metaspace:** class metadata (replaced PermGen in Java 8).
- GC roots $\rightarrow$ reachability; unreachable objects collected. finalize() deprecated.

Java Pitfalls (favorite trick questions)

- == compares references for objects; use .equals(). But Integer caches -128..127, so Integer a=127, b=127; a==b is true, while 128 gives false.
- String concatenation in loops $\rightarrow$ O(n²); use StringBuilder.
- Autoboxing NPE: unboxing a null Integer throws NullPointerException.
- Floating point: never compare double with ==; use an epsilon or BigDecimal for money (enterprise/Oracle systems especially).

2. Discrete Mathematics

Logic

- Connectives: $\neg$, $\wedge$, $\vee$, $\rightarrow$ (implication), $\leftrightarrow$ (biconditional), $\oplus$ (XOR).
- $p \rightarrow q$ is false **only** when p is true and q is false. Equivalent to $\neg p \vee q$.
- Converse: $q \rightarrow p$ · Inverse: $\neg p \rightarrow \neg q$ · **Contrapositive: $\neg q \rightarrow \neg p$ ($\equiv$ original)** — converse/inverse are NOT equivalent to original (common trap).
- De Morgan: $\neg(p \wedge q) \equiv \neg p \vee \neg q$; $\neg(p \vee q) \equiv \neg p \wedge \neg q$.
- Tautology = always true; contradiction = always false; contingency = depends.
- Quantifier negation: $\neg \forall x P(x) \equiv \exists x \neg P(x)$; $\neg \exists x P(x) \equiv \forall x \neg P(x)$. Order matters: $\forall x \exists y \neq \exists y \forall x$.
- Inference rules: Modus Ponens ($p, p \rightarrow q \vdash q$), Modus Tollens ($\neg q, p \rightarrow q \vdash \neg p$), Hypothetical Syllogism ($p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$).

Set Theory

- $|A \cup B| = |A| + |B| - |A \cap B|$ (**inclusion–exclusion**; extend for 3 sets: $+|A \cap B \cap C|$ back).
- Power set of n -element set: 2^n subsets. Cartesian product $|A \times B| = |A||B|$.
- $A - B = A \cap B^c$. Symmetric difference $A \triangle B = (A \cup B) - (A \cap B)$.

Relations & Functions

Property	Definition
Reflexive	$(a,a) \in R \ \forall a$
Symmetric	$(a,b) \in R \Rightarrow (b,a) \in R$
Antisymmetric	$(a,b), (b,a) \in R \Rightarrow a=b$
Transitive	$(a,b), (b,c) \in R \Rightarrow (a,c) \in R$

- Equivalence relation** = reflexive + symmetric + transitive $\rightarrow$ partitions the set into equivalence classes.
- Partial order** = reflexive + antisymmetric + transitive (visualized by Hasse diagram).
- Functions: injective (one-to-one), surjective (onto), bijective (both $\Rightarrow$ invertible).
- Number of functions $A \rightarrow B$: $|B|^{|A|}$; injections: $P(|B|, |A|)$; bijections ($|A|=|B|=n$): $n!$.
- Pigeonhole principle**: $n+1$ items into n boxes $\Rightarrow$ some box has ≥ 2 . Generalized: $\lceil n/k \rceil$.

Counting

Formula	Use
$P(n,r) = n!/(n-r)!$	ordered selection
$C(n,r) = n!/(r!(n-r)!)$	unordered selection
n^r	selection with repetition, ordered
$C(n+r-1, r)$	multiset selection (stars & bars)
Circular permutations = $(n-1)!$	round table

- Binomial theorem: $(x+y)^n = \sum C(n,k) x^{n-k} y^k$. $\sum C(n,k) = 2^n$.
- Derangements: $D(n) = n! \sum (-1)^k/k! \approx n!/e$.

Recurrences & Induction

- Linear homogeneous: solve characteristic equation. Fibonacci: $r^2 = r + 1 \rightarrow$ closed form via golden ratio φ .
- Induction: base case + (inductive hypothesis $\Rightarrow$ step). Strong induction assumes all smaller cases.

Graph Theory (math side)

- Handshake theorem: $\sum \deg(v) = 2|E| \Rightarrow$ number of odd-degree vertices is even.
- **Euler circuit**: connected + every vertex even degree. Euler path: exactly 0 or 2 odd vertices.
- **Hamiltonian** path/circuit: visits every *vertex* once — no easy criterion (NP-complete). Euler = edges, Hamilton = vertices (mnemonic: **Euler–Edges**).
- Complete graph K_n : $n(n-1)/2$ edges. Tree with n vertices: exactly $n-1$ edges, connected, acyclic.
- Planar graphs: Euler's formula $v - e + f = 2$; planar $\Rightarrow e \leq 3v - 6$. K_5 and $K_{3,3}$ are non-planar (Kuratowski).
- Bipartite $\Leftrightarrow$ no odd cycles $\Leftrightarrow$ 2-colorable. Four-color theorem: planar maps need ≤ 4 colors.

Number Theory (crypto foundation)

- gcd via Euclid: $\gcd(a,b) = \gcd(b, a \bmod b)$. Extended Euclid gives $ax + by = \gcd(a,b)$.
 - $a \equiv b \pmod{m} \Leftrightarrow m \mid (a-b)$. Modular inverse exists $\Leftrightarrow \gcd(a,m)=1$.
 - **Fermat's little theorem**: $a^{p-1} \equiv 1 \pmod{p}$ for prime p , $p \nmid a$.
 - Euler's theorem: $a^{\varphi(n)} \equiv 1 \pmod{n}$; $\varphi(p) = p-1$, $\varphi(pq) = (p-1)(q-1)$ — the basis of **RSA**.
 - Chinese Remainder Theorem: unique solution mod product of pairwise-coprime moduli.
-

3. Digital Logic Design

Number Systems

- Conversions: binary $\leftrightarrow$ octal (group 3 bits), binary $\leftrightarrow$ hex (group 4 bits).
- **Two's complement** (standard for signed): invert bits + 1. Range for n bits: $-2^{n-1} \dots 2^{n-1}-1$ (one more negative number than positive!).
- Overflow detection (addition): operands same sign but result differs; or carry-in $\neq$ carry-out at MSB.
- BCD: each decimal digit in 4 bits; add 6 (0110) correction when digit sum > 9 .
- Gray code: successive values differ in 1 bit (binary $\rightarrow$ Gray: $g = b \oplus (b \gg 1)$).

Boolean Algebra

Law	Form
Identity	$A+0=A, A \cdot 1=A$
Null	$A+1=1, A \cdot 0=0$
Idempotent	$A+A=A$
Complement	$A+A'=1, A \cdot A'=0$
Absorption	$A + AB = A; A + A'B = A + B$
De Morgan	$(AB)' = A' + B'; (A+B)' = A'B'$
Consensus	$AB + A'C + BC = AB + A'C$

- **NAND and NOR are universal gates** (each alone can build any circuit).
- XOR: $A \oplus B = A'B + AB'$; parity, adders, comparators. XOR with 1 inverts; with 0 passes through.
- SOP (sum of products) $\leftrightarrow$ minterms (m); POS $\leftrightarrow$ maxterms (M). n variables $\rightarrow 2^n$ minterms.

Karnaugh Maps

- Group 1s in powers of 2 (1, 2, 4, 8...); groups can wrap around edges; bigger groups = simpler terms.
- "Don't care" (X) cells: use them if they enlarge a group.
- Prime implicant = maximal group; essential PI = covers a minterm no other PI covers. Quine–McCluskey = tabular method for >4 variables.

Combinational Circuits

Circuit	Function
Half adder	$S = A \oplus B$, $C = AB$
Full adder	$S = A \oplus B \oplus C_{in}$, $C_{out} = AB + C_{in}(A \oplus B)$
Multiplexer ($2^n:1$)	selects 1 of 2^n inputs with n select lines; can implement any n -var function
Demux / Decoder	$1 \rightarrow 2^n$ routing / $n \rightarrow 2^n$ line activation
Encoder / Priority encoder	$2^n \rightarrow n$; priority resolves multiple inputs
Comparator	$A > B$, $A = B$, $A < B$ outputs

- Ripple-carry adder delay grows $O(n)$; **carry-lookahead** computes carries in parallel ($G = AB$, $P = A \oplus B$).

Sequential Circuits

Flip-flop	Characteristic	Notes
SR	$S=1, R=1$ invalid	set/reset
D	$Q^+ = D$	data latch, most common
JK	$J=K=1$ toggles	fixes SR invalid state
T	$T=1$ toggles	counters

- Latch = level-triggered; flip-flop = edge-triggered** (master-slave beats race conditions).
- Setup time (data stable *before* clock edge) & hold time (*after*); violations $\rightarrow$ metastability.
- Counters: ripple/asynchronous (simple, slow, glitches) vs synchronous (common clock). n flip-flops $\rightarrow$ mod- 2^n counter; decade counter = mod-10.
- Registers: SISO, SIPO, PISO, PIPO shift registers; ring counter (n states), Johnson counter ($2n$ states).

FSM: Mealy vs Moore

	Moore	Mealy
Output depends on	state only	state + input
States needed	usually more	usually fewer
Output timing	synchronized with clock	can glitch on input change

- Design flow: state diagram $\rightarrow$ state table $\rightarrow$ state assignment $\rightarrow$ excitation table $\rightarrow$ K-maps $\rightarrow$ circuit.

4. Numerical Methods

Errors

- Absolute error = $|\text{true} - \text{approx}|$; relative error = $\text{absolute} / |\text{true}|$.
- Round-off error** (finite precision, IEEE 754) vs **truncation error** (cutting infinite process, e.g., Taylor series).
- Significant digits; machine epsilon $\approx 2.22 \times 10^{-16}$ for double.
- Pitfall**: subtracting nearly equal numbers $\rightarrow$ catastrophic cancellation.

Root Finding

Method	Formula / idea	Convergence	Notes
Bisection	halve $[a,b]$ where $f(a)f(b)<0$	linear (guaranteed)	slow but always converges; error $\leq (b-a)/2^n$
Regula Falsi	secant line through bracket	superlinear-ish, keeps bracket	one endpoint can stagnate
Secant	$x_{n+1} = x_n - f(x_n)(x_n - x_{n-1}) / (f(x_n) - f(x_{n-1}))$	order ≈ 1.618	no derivative needed
Newton–Raphson	$x_{n+1} = x_n - f(x_n)/f'(x_n)$	quadratic	fails if $f' \approx 0$, bad start diverges
Fixed point	$x = g(x)$ iteration	linear if	$g'(x)$

Linear Systems

- Direct: **Gaussian elimination** $O(n^3)$; partial pivoting for stability; LU decomposition (reuse for multiple b vectors); Cholesky for symmetric positive definite.
- Iterative: **Jacobi** (all updates from old values) vs **Gauss–Seidel** (use fresh values immediately $\rightarrow$ usually $\sim 2\times$ faster). Converge if matrix is **strictly diagonally dominant**.
- Ill-conditioned system: large condition number $\rightarrow$ small input changes blow up the solution.

Interpolation

- **Lagrange**: $P(x) = \sum y_i \prod (x-x_j)/(x_i-x_j)$ — no table, but recompute for new points.
- **Newton's divided differences**: incremental, add points easily. Forward/backward differences for equally spaced data.
- Error term involves $f^{(n+1)}(\xi)/(n+1)!$; high-degree polynomials oscillate (**Runge phenomenon**) $\rightarrow$ prefer splines (cubic spline: C^2 continuity).

Numerical Integration (h = interval width)

Rule	Formula	Error order
Trapezoidal	$h/2 [f_0 + 2f_1 + \dots + f_n]$	$O(h^2)$
Simpson's 1/3	$h/3 [f_0 + 4f_{\text{odd}} + 2f_{\text{even}} + f_n]$ — needs even # of intervals	$O(h^4)$
Simpson's 3/8	$3h/8 [f_0 + 3f_1 + 3f_2 + 2f_3 \dots]$ — intervals multiple of 3	$O(h^4)$

- Mnemonic for Simpson 1/3 coefficients: **1-4-2-4-2-...-4-1**.

ODEs ($y' = f(x,y)$)

Method	Update	Local error
Euler	$y_{n+1} = y_n + h \cdot f(x_n, y_n)$	$O(h^2)$ (global $O(h)$)
Modified Euler / Heun	average of slopes at both ends	$O(h^3)$
RK4	weighted average $k_1..k_4$: $y_{n+1} = y_n + h/6 (k_1 + 2k_2 + 2k_3 + k_4)$	$O(h^5)$ (global $O(h^4)$)

- RK4 = default workhorse: accuracy vs cost sweet spot.

5. Automata and Computability

Chomsky Hierarchy (memorize this table)

Type	Grammar	Language	Machine
3	Regular	Regular	Finite Automaton (DFA/NFA)
2	Context-Free	CFL	Pushdown Automaton (PDA)
1	Context-Sensitive	CSL	Linear Bounded Automaton
0	Unrestricted	Recursively Enumerable	Turing Machine

- Strict containment: $\text{Regular} \subset \text{CFL} \subset \text{CSL} \subset \text{RE}$.

Finite Automata & Regular Languages

- DFA: exactly one transition per (state, symbol). NFA: multiple/ ϵ -moves allowed. **NFA $\equiv$ DFA in power**; subset construction may cause **2^n state blow-up**.
- Regular expressions $\equiv$ finite automata (Kleene's theorem). Conversions: $\text{RE} \rightarrow \text{NFA}$ (Thompson), $\text{NFA} \rightarrow \text{DFA}$ (subset), $\text{DFA} \rightarrow \text{minimal DFA}$ (partition refinement / Myhill–Nerode).
- Closure: regular languages closed under union, concatenation, star, intersection, complement, reversal.
- **Pumping lemma (regular)**: for long enough $w = xyz$, $|xy| \leq p$, $|y| \geq 1$, $xy^iz \in L \ \forall i$. Used to prove NON-regularity (e.g., $a^n b^n$). *Cannot prove a language regular*.
- Classic non-regular: $\{a^n b^n\}$, palindromes, balanced parentheses (need counting/stack).

CFGs & PDAs

- CFG: V, Σ, R, S ; derivations; leftmost/rightmost; parse trees. **Ambiguous** = string with 2 parse trees (e.g., dangling else). No algorithm decides CFG ambiguity (undecidable).
- Chomsky Normal Form: $A \rightarrow BC$ or $A \rightarrow a$ (needed for CYK parsing, $O(n^3)$).
- PDA = NFA + stack; accept by final state or empty stack (equivalent). **Deterministic PDA weaker than nondeterministic** (unlike FA!) — e.g., even palindromes need NPDA.
- CFL closure: union, concat, star YES; **intersection & complement NO** (but $\text{CFL} \cap \text{regular} = \text{CFL}$).
- Pumping lemma (CFL): $uvxyz$ form; proves $\{a^n b^n c^n\}$ not context-free.

Turing Machines & Decidability

- TM = FA + infinite tape + read/write head both directions. Variants (multi-tape, nondeterministic) equal in power. **Church–Turing thesis**: TM $\equiv$ “effectively computable.”
- **Recursive (decidable)**: TM halts on all inputs with accept/reject. **Recursively enumerable**: TM accepts members, may loop on non-members. $L \text{ decidable} \Leftrightarrow L \text{ and } \bar{L} \text{ both RE}$.
- **Halting problem is undecidable** (diagonalization). Rice's theorem: every non-trivial *semantic* property of RE languages is undecidable.
- Reductions: $A \leq B$ and $A \text{ undecidable} \Rightarrow B \text{ undecidable}$ (reduce FROM the known-hard problem — direction is the classic mistake).
- Decidable examples: DFA membership/emptiness/equivalence, CFG membership. Undecidable: TM halting, CFG equivalence/ambiguity, PCP.

6. Computer Architecture

Performance

- **CPU time** = **Instruction Count** × **CPI** × **Clock cycle time**. MIPS is misleading across ISAs.
- **Amdahl's Law**: $\text{Speedup} = 1 / ((1-f) + f/s)$ — the serial fraction caps everything. (f = enhanced fraction, s = enhancement speedup.)

ISA & CPU Organization

	RISC	CISC
Instructions	simple, fixed-length, load/store only touch memory	complex, variable length, memory operands
CPI	~1, pipeline-friendly	multi-cycle
Examples	ARM, MIPS, RISC-V	x86

- Addressing modes: immediate, register, direct, indirect, register-indirect, indexed/displacement, base, relative.
- Instruction cycle: Fetch → Decode → Execute (→ Memory → Write-back). Components: PC, IR, MAR, MDR, ALU, register file, control unit (hardwired = fast, microprogrammed = flexible).

Pipelining

- Classic 5 stages: **IF** – **ID** – **EX** – **MEM** – **WB**. Ideal speedup = number of stages.
- Hazards:
 - **Structural** — resource conflict (fix: duplicate hardware, separate I/D caches).
 - **Data** — RAW (true dependency; fix: **forwarding/bypassing**, stall for load-use), WAR/WAW (name deps; register renaming).
 - **Control** — branches (fix: branch prediction, delayed branch, flush on mispredict).
- Deeper pipelines raise frequency but increase hazard/mispredict penalties.

Memory Hierarchy & Cache

- Registers → L1 → L2 → L3 → RAM → disk. Works because of **temporal & spatial locality**.
- Mapping: **direct** (1 place, cheap, conflict misses), **fully associative** (anywhere, expensive), **k-way set associative** (compromise).
- Address split: tag | set index | block offset.
- **AMAT** = **hit time** + **miss rate** × **miss penalty**.
- Miss types (**3 Cs**): Compulsory (first touch), Capacity (cache too small), Conflict (mapping collisions).
- Write policies: write-through (+write buffer) vs **write-back** (dirty bit); write-allocate vs no-write-allocate.
- Replacement: LRU, FIFO, random.
- Cache line size trade-off: bigger lines exploit spatial locality but raise miss penalty & pollution.

Parallelism & I/O

- Flynn's taxonomy: SISD, **SIMD** (GPU/vector), MISD (rare), **MIMD** (multicore).
- ILP: superscalar (multiple issue), out-of-order execution, branch prediction, speculation.
- Multicore cache coherence: MESI protocol (Modified/Exclusive/Shared/Invalid); false sharing hurts performance.
- I/O: programmed I/O (polling) < interrupt-driven < **DMA** (device transfers directly to memory, CPU only at start/end).

- Endianness: little-endian (x86) stores LSB at lowest address; big-endian (network byte order).

7. Compiler Design

Phases (front end → back end)

Source → Lexical Analysis → Syntax Analysis → Semantic Analysis
 → Intermediate Code Gen → Code Optimization → Code Generation → Target
 (Symbol table + error handler run alongside all phases)

- Compiler (translate whole program ahead) vs interpreter (execute line-by-line) vs hybrid (Java: compile to bytecode, JVM interprets/JIT).
- Cross-compiler targets a different machine; bootstrapping = compiler written in its own language.

Lexical Analysis

- Tokens (category), lexemes (actual text), patterns (regex). Whitespace/comments stripped here.
- Implemented with regex → NFA → DFA (lex/flex). Longest-match + rule-priority disambiguation.
- Errors here: invalid characters, malformed literals.

Parsing

	Top-down (LL)	Bottom-up (LR)
Builds	leftmost derivation	rightmost derivation in reverse
Direction	expand from start symbol	shift/reduce to start symbol
Grammar prep	remove left recursion , left-factor	handles left recursion fine
Power	$LL(1) \subset$	$SLR \subset LALR \subset LR(1)$
Tools	recursive descent (hand-written)	yacc/bison (LALR)

- Left recursion removal: $A \rightarrow A\alpha \mid \beta$ becomes $A \rightarrow \beta A'$, $A' \rightarrow \alpha A' \mid \epsilon$.
- **FIRST** = terminals that can begin derivations; **FOLLOW** = terminals that can appear after a nonterminal.
 $LL(1)$ table conflict $\Rightarrow$ grammar not $LL(1)$.
- LR conflicts: shift-reduce (classic: dangling else — resolve by preferring shift) and reduce-reduce.
- Ambiguity fixed by precedence & associativity declarations or grammar rewriting.

Semantic Analysis & Intermediate Code

- Type checking, scope resolution via **symbol table** (hash table per scope, chained).
- Syntax-Directed Definitions: **synthesized** attributes (from children — S-attributed, fits LR) vs **inherited** (from parent/siblings — L-attributed).
- IR forms: AST, **three-address code** ($x = y \text{ op } z$), quadruples/triples, static single assignment (SSA).

Optimization (machine-independent)

Technique	Example
Constant folding	$3*4 \rightarrow 12$ at compile time
Constant propagation	$x=5; y=x+1 \rightarrow y=6$
Common subexpression elimination	compute $a+b$ once

Technique	Example
Dead code elimination	remove unreachable/unused
Strength reduction	$x*2 \rightarrow x\ll 1$; loop mult $\rightarrow$ add
Loop-invariant code motion	hoist constant work out of loop
Loop unrolling	fewer branches, more ILP
Peephole	local pattern rewrites on target code

- Basic block = straight-line code, one entry one exit; control-flow graph (CFG) over blocks; data-flow analysis (liveness $\rightarrow$ register allocation via graph coloring).

Runtime & Codegen

- Activation record (stack frame): return address, parameters, locals, saved registers, dynamic/static links.
- Static vs stack vs heap allocation; parameter passing: by value, by reference, (by name — theory).
- Code generation issues: instruction selection, register allocation, evaluation order.

8. Computer Graphics

Fundamentals

- Raster (pixels, resolution-bound) vs vector (equations, scale-free). Frame buffer holds the pixel image; refresh rate vs frame rate; **double buffering** kills flicker.
- Color: RGB (additive, screens), CMYK (subtractive, print), HSV (intuitive picking). Color depth: 24-bit = 8 bits/channel.
- Aliasing = jaggies from sampling; anti-aliasing = supersampling / area averaging.

Scan Conversion

- **DDA line**: increment by slope, uses floats (simple but slow/rounding).
- **Bresenham line**: integer-only decision parameter — the standard. Initial $d = 2\Delta y - \Delta x$ (for $|m| < 1$).
- **Midpoint circle**: exploit 8-way symmetry, decision $p_0 = 1 - r$.
- Polygon fill: scan-line algorithm (edge table, active edge list, parity rule), flood fill (any interior color) vs boundary fill (until border color).

2D/3D Transformations (homogeneous coordinates — the reason: makes translation a matrix multiply)

Transform	2D matrix (3×3) essentials
Translation	last column = (tx, ty, 1)
Scaling	diag(sx, sy, 1)
Rotation (about origin)	$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$
Reflection	negative scale
Shear	off-diagonal terms

- Rotation about arbitrary point P: **translate P $\rightarrow$ origin, rotate, translate back** ($T^{-1} \cdot R \cdot T$ applied right-to-left). Matrix order matters — transformations compose right-to-left; matrix multiplication is not commutative.
- 3D adds z; rotations about x/y/z axes; 4×4 matrices.

Viewing Pipeline

Model coords → World → View/Camera → Projection → Clip → NDC → Viewport (screen)

- **Parallel/orthographic projection** (preserves parallelism, CAD) vs **perspective projection** (foreshortening, realism; farther = smaller; defined by center of projection).
- Clipping: **Cohen–Sutherland** line clipping (4-bit region outcodes: trivially accept if both 0000, trivially reject if AND $\neq 0$), Liang–Barsky (parametric, fewer intersections), Sutherland–Hodgman polygon clipping (clip against each window edge).

Visibility & Shading

- Hidden surface removal: **Z-buffer** (per-pixel depth compare — simple, memory-hungry, the hardware standard), painter's algorithm (sort back-to-front; fails on cyclic overlap), back-face culling ($\text{normal} \cdot \text{view} > 0$).
- Illumination = ambient + diffuse (Lambert: $N \cdot L$) + specular (Phong: $(R \cdot V)^n$).
- Shading: **Flat** (one color/polygon) < **Gouraud** (interpolate vertex colors — misses small highlights) < **Phong shading** (interpolate normals, light per pixel — better, costlier).
- Curves: Bézier (control points, Bernstein basis; curve inside convex hull; C(3) cubic most common) vs B-splines (local control, higher continuity). Fractals for natural forms.

9. Software Engineering

Process Models

Model	Idea	Use when / weakness
Waterfall	linear phases: req → design → build → test → deploy → maintain	stable requirements; change is expensive, late testing
V-Model	each dev phase paired with a test phase	safety-critical; same rigidity
Incremental	deliver in slices	early partial delivery
Spiral	iterations driven by risk analysis	large risky projects; costly
Agile/Scrum	sprints (1–4 wks), product backlog, daily stand-up, sprint review + retrospective; roles: PO, Scrum Master, dev team	changing requirements; needs customer engagement
Kanban	continuous flow, WIP limits	ops/maintenance work

- Agile Manifesto: individuals & interactions > processes; working software > docs; customer collaboration > contracts; responding to change > plan.

Requirements & Design

- Functional (what it does) vs **non-functional** (quality: performance, security, scalability, usability — often forgotten in exams and real projects alike).
- SRS qualities: correct, unambiguous, complete, verifiable, consistent, traceable.
- UML: use case, class, sequence, activity, state, component, deployment diagrams.
- **Cohesion high + coupling low = good design** (mnemonic: “maximize what belongs together, minimize what ties modules together”).

- Cohesion (worst→best): coincidental → logical → temporal → procedural → communicational → sequential → **functional**.
- Coupling (worst→best): content → common → control → stamp → **data**.

SOLID Principles

Letter	Principle	One-liner
S	Single Responsibility	one reason to change
O	Open/Closed	open to extension, closed to modification
L	Liskov Substitution	subtypes must be usable as their base type
I	Interface Segregation	many small interfaces > one fat one
D	Dependency Inversion	depend on abstractions, not concretions

- Other: DRY, KISS, YAGNI, separation of concerns.

Design Patterns (GoF highlights)

- **Creational**: Singleton (one instance — beware thread safety & testability), Factory Method, Abstract Factory, Builder, Prototype.
- **Structural**: Adapter (interface converter), Decorator (add behavior dynamically), Facade (simple front), Proxy, Composite.
- **Behavioral**: Observer (publish/subscribe), Strategy (swap algorithms), Command, Iterator, State, Template Method.
- Architecture: MVC (model/view/controller), layered (presentation/business/data — typical enterprise app), client-server, microservices vs monolith (independent deploys vs operational complexity), event-driven.

Testing

Level	Scope
Unit	single function/class (JUnit; mocks/stubs isolate dependencies)
Integration	modules together (big-bang vs incremental: top-down w/ stubs, bottom-up w/ drivers)
System	whole product vs requirements
Acceptance (UAT)	customer sign-off; alpha (in-house) / beta (real users)

- **Black-box** (no code knowledge): equivalence partitioning, **boundary value analysis** (bugs love edges), decision tables. **White-box**: statement/branch/path coverage; cyclomatic complexity $V(G) = E - N + 2$ = independent paths = decisions + 1.
- Verification (“building the product right” — reviews, static analysis) vs Validation (“building the right product” — testing against needs).
- Regression testing after changes; smoke test = build sanity; TDD = red → green → refactor.
- A defect found in production costs orders of magnitude more than in requirements — argue for early testing.

Estimation, Maintenance & DevOps

- COCOMO: effort = $a \cdot (KLOC)^b$ (organic/semi-detached/embedded); Function Points = size from user-visible functionality.

- Maintenance types: **corrective** (bugs), **adaptive** (environment changes), **perfective** (enhancements — the biggest share), **preventive** (refactoring).
 - Version control (Git): branch, merge vs rebase, pull request review. **CI/CD**: automated build + test on each commit; continuous delivery/deployment.
 - Technical debt: shortcuts now, interest later. Code review + static analysis + refactoring keep it payable.
 - Risk management: identify → analyze (probability × impact) → mitigate/monitor (RMMM).
-

Last-Minute Cross-Subject Hit List

- `equals()`/`hashCode()` contract and overloading vs overriding — the two most-asked Java questions.
- Two's complement range (one extra negative number) and overflow detection ↔ digital logic **and** computer architecture.
- Boolean algebra / De Morgan ↔ digital logic simplification **and** propositional logic in discrete math.
- Finite automata ↔ automata theory, lexical analysis (compilers), and regex — one topic, three exams.
- Pumping lemma proves NON-membership only (can't prove a language regular/context-free) — direction matters, same as reduction direction in undecidability proofs.
- Chomsky hierarchy ↔ automata **and** the parsing power ladder in compilers ($LL(1) \subset SLR \subset LALR \subset LR(1)$).
- Fermat/Euler theorems (discrete math) → RSA — expect one crypto-flavored number theory question.
- Amdahl's Law and the 3 pipeline hazard types ↔ the two most predictable architecture questions.
- Newton–Raphson = quadratic convergence but needs $f' \neq 0$; Simpson $1/3 = \mathbf{1-4-2-\dots-4-1}$ with an even number of intervals.
- Transformation composition is right-to-left (graphics) — same non-commutativity trap as matrix chain questions anywhere.
- Cohesion high / coupling low + SOLID ↔ the design-quality vocabulary interviewers expect in any code discussion.